

CMSC 25300/35300, STAT 27700 (Fall 2024)

Homework 5

Submission Instructions:

- Please submit your homework in PDF to Gradescope (which can be accessed from the course's website on Canvas). Make sure to match all questions to the correct pages in Gradescope - points will be deducted if these are not properly matched.
- Please paste your code in the submitted PDF. In other words, your submission should be a single PDF that contains both your writing solutions and your code. Be sure to include the output of your code when relevant (e.g. text and figures). Keep in mind that we are not easily able to run your code.
- Note that you do not need to copy the problem statements in your solution, *as long as you clearly indicate the problem numbers* (e.g., 1.a, 2.c, etc).

1. **The SVD and Least Squares.** Let $\mathbf{X} \in \mathbb{R}^{n \times d}$ with $\text{rank}(\mathbf{X}) = k < \min\{n, d\}$. Each row represents one training sample, and each column represents one feature. Let $\mathbf{y} \in \mathbb{R}^n$ be the response vector for each sample. The full SVD of $\mathbf{X}$ is given by $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, where

$$\mathbf{U} = \begin{bmatrix} | & & | & & | \\ \mathbf{u}_1 & \cdots & \mathbf{u}_k & \cdots & \mathbf{u}_n \\ | & & | & & | \end{bmatrix}, \quad \mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_k & & \\ & & & 0 & \\ & & & & \ddots \end{bmatrix}, \quad \mathbf{V}^T = \begin{bmatrix} - & \mathbf{v}_1^T & - \\ & \vdots & \\ - & \mathbf{v}_k^T & - \\ & \vdots & \\ - & \mathbf{v}_d^T & - \end{bmatrix}$$

- a.
 - i) (2 points) Write $\mathbf{X}$ as the sum of rank 1 matrices.
 - ii) (3 points) Give expressions for the SVDs of $\mathbf{X}^\top$, $\mathbf{X}\mathbf{X}^\top$ and $\mathbf{X}^\top\mathbf{X}$.
- b. Now we focus on the least squares problem, and try to find a weight vector $\mathbf{w}$ by minimizing $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$.

- i) (5 points) Consider a simple case where $\mathbf{X} = \begin{bmatrix} 3 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$ and $\mathbf{y} = \begin{bmatrix} 6 \\ 2 \\ 1 \\ 2 \end{bmatrix}$. We

would like to find $\mathbf{w}$ which minimizes the squared error, i.e. $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$. However, we know that there is no such unique $\mathbf{w}$ because $\mathbf{X}$ does not have full column rank. Describe the set of $\mathbf{w}$ vectors which minimize the squared error. Which of these vectors has the smallest norm?

- ii) (4 points) Now we return to the general case, with $\mathbf{X}$ and $\mathbf{y}$ unknown, and with the SVD of $\mathbf{X}$ given by $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. Find a basis for all weight vectors that satisfy $\mathbf{X}\mathbf{w} = 0$.
- iii) (5 points) Let $\tilde{\mathbf{y}} = \mathbf{U}^\top \mathbf{y}$ and $\tilde{\mathbf{w}} = \mathbf{V}^\top \mathbf{w}$. Prove that $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 = \|\tilde{\mathbf{y}} - \mathbf{\Sigma}\tilde{\mathbf{w}}\|_2^2$. Recall your solution from part (i), and give the set of $\tilde{\mathbf{w}}$ which minimizes the norm above. Which of these has the smallest norm?
- iv) (4 points) Given the $\tilde{\mathbf{w}}$ you found above, what is the $\mathbf{w}$ with minimum norm which is a solution to the original least squares problem?
- c. Least squares estimates can be used in many contexts. Consider a signal, $\mathbf{y}$, which is known to depend on a transformation matrix $\mathbf{X}$, an input vector $\mathbf{w}$, and some noise $\mathbf{e}$ as follows:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \mathbf{e}.$$

Suppose that we know $\mathbf{X}$, and observe $\mathbf{y}$, but we do not know $\mathbf{w}$. Then, we could use least-squares to estimate $\mathbf{w}$ from the known values $\mathbf{y}$ and $\mathbf{X}$. Use the starter code below to load the data, and then write code to answer the following questions.

```
import numpy as np
import matplotlib.pyplot as plt

# Load data
data = np.load("blurring.npz")
X = data['X']
y = data['y']

U, S, Vt = np.linalg.svd(X, full_matrices=False)

# Part 1.c.i.
# Estimate w using X and y with regular least squares
# Your code here

w_LS =

# Plot true signal, and LS estimate
plt.plot(data['w'], label='true w')
plt.plot(w_LS, alpha=0.7, c='red', label='LS estimate')
plt.legend()
plt.show()

# Part 1.c.ii.
# Estimate w using X and y with truncated SVD
# Your code here

w_15 =
w_75 =
w_200 =

# Plot true signal, and truncated SVD estimates
```

```
plt.plot(data['w'], label='true_w')
plt.plot(w_15, alpha=0.7, c='red', label='k=15')
plt.plot(w_75, alpha=0.7, c='orange', label='k=75')
plt.plot(w_200, alpha=0.7, c='purple', label='k=200')
plt.legend()
plt.show()
```

- i. (6 points) Use standard least squares to obtain an estimate of $\mathbf{w}$ and compare the true $\mathbf{w}$ to your approximation using the plotting code above. What is the impact of the added noise to your estimate?
 - ii. (6 points) Use the truncated SVD to obtain three estimates of $\mathbf{w}$ (with $k = 15, 75, 200$) and compare the true $\mathbf{w}$ to your approximates. How does regularization using the SVD compare to least squares? How do the different values of k impact the approximation?
2. **PageRank.** Let us consider a web of n pages. Imagine we surf the web by randomly clicking on links at each web page when you arrive at it. If we click randomly in this way long enough, we'll reach a steady state where π_i is the probability that we're at page i at any given time. That is, you will end up landing on “important” pages more frequently, where “importance” reflects the number of other web pages linking to it, weighted by the importance of those pages. We wish to find a vector of probabilities $\boldsymbol{\pi} = [\pi_1, \dots, \pi_n]^\top$ reflecting the steady state behavior.

Define an $n \times n$ adjacency matrix $\mathbf{M}$, corresponding to n pages as

$$M_{ij} = \begin{cases} 1, & \text{if page } j \text{ has a link to page } i \\ 0, & \text{otherwise} \end{cases}$$

Then we define the “network adjacency matrix” $\mathbf{A}$ as the column normalized version of $\mathbf{M}$. The PageRank vector $\boldsymbol{\pi}$ satisfies

$$\boldsymbol{\pi} = \mathbf{A}\boldsymbol{\pi}.$$

That is, $\boldsymbol{\pi}$ is an *eigenvector* of $\mathbf{A}$.

Given a matrix $\mathbf{A}$, we say a non-zero vector $\mathbf{v}$ is an *eigenvector* of $\mathbf{A}$ if $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ for some scalar λ , which is called an *eigenvalue*. Eigenvectors have important connections to singular vectors. Consider a matrix and its singular value decomposition $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. Then, if $\mathbf{A} = \mathbf{X}^\top\mathbf{X}$, we have $\mathbf{A} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^\top$. Multiplying both sides of this equation on the right by $\mathbf{V}$ gives $\mathbf{A}\mathbf{V} = \mathbf{V}\mathbf{\Sigma}^2$, or $\mathbf{A}\mathbf{v}_i = \sigma_i^2\mathbf{v}_i$. That is, the right singular vectors of $\mathbf{X}$ are the eigenvectors of $\mathbf{A}$ and the singular values of $\mathbf{X}$ can be squared to give the eigenvalues of $\mathbf{A}$ corresponding eigenvalues of $\mathbf{A}$.

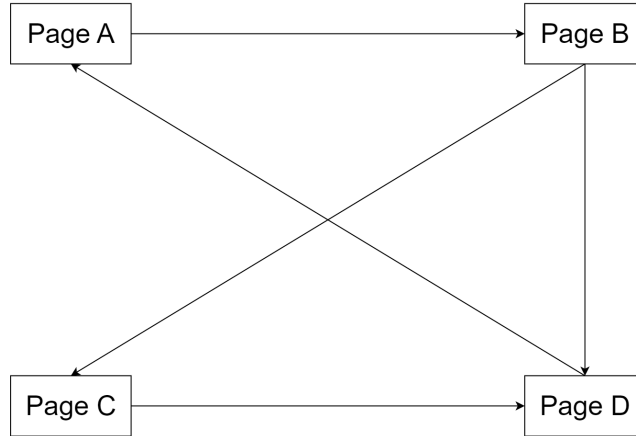
When each column of $\mathbf{A}$ sums to 1, we can compute the PageRank vector $\boldsymbol{\pi} \in \mathbb{R}^n$ by repeatedly multiplying by $\mathbf{A}$ until convergence (and normalize norm of $\boldsymbol{\pi}^{(t)}$ along the way):

$$\boldsymbol{\pi}^{(1)} = \mathbf{A}\boldsymbol{\pi}^{(0)}, \boldsymbol{\pi}^{(2)} = \mathbf{A}\boldsymbol{\pi}^{(1)}, \dots, \boldsymbol{\pi}^{(t)} = \mathbf{A}\boldsymbol{\pi}^{(t-1)}$$

The elements of the converged vector π are the PageRank of corresponding pages, and the matrix A is also called “Google matrix”.

Note that if we can find the converged page rank vector π , π must be an eigenvector of the matrix A with eigenvalue one.

- a. (5 points) Consider the following network:



What is M for this network? A ?

- b. (8 points) Using this matrix A , what is the PageRank of each of the 4 pages? Write a Python script to compute this.
- c. (8 points) In real application, Google found it useful to add a “damping factor” α such that $0 < \alpha < 1$. The modified “Google matrix” G is then defined to be

$$G = \alpha A + (1 - \alpha) \mathbf{u} \mathbf{1}^T$$

where $\mathbf{1} \in \mathbb{R}^n$ is all-ones vector and $\mathbf{u}$ models the likelihood of users clicking a page (whether or not there is a link to it on current page). Theoretically, we could use any vector $\mathbf{u} \in \mathbb{R}^n$, and in practice Google used $\mathbf{u} = \mathbf{1}$, so that

$$G = \alpha A + (1 - \alpha) \frac{1}{n} \mathbf{1} \mathbf{1}^T.$$

Intuitively, you can think of the damped PageRank as follows: a user who clicks a random link on the current page with probability α , but with probability $1 - \alpha$ the user would choose a uniformly random page (whether or not there is a link to it).

For $\alpha = 0.8$, what is the PageRank of each of the 4 pages? Write a Python script to compute this. (HINT: If you have a matrix G and want to normalize it so each column sums to 1, use the line `G = G/G.sum(axis=0).`)

- d. Consider a world of n pages, in which every page (including Youtube and Wikipedia) links to Youtube and Wikipedia, and no page links anywhere else. Youtube is page 1, Wikipedia is page 2, and the other $n - 2$ pages are page 3 to n .
- i) (4 points) What is the adjacency matrix M ? With a damping parameter α , what is the Google matrix G ?

- ii) (5 points) If the PageRank vector π is a probability vector in $\mathbb{R}^n$ (elements sum up to 1), what is the PageRank of Youtube and Wikipedia? What about the rank of the rest $n - 2$ pages? Express these two ranks in terms of damping parameter α and total number of pages n . (HINT: let $\pi = [x, x, y, \dots, y]^T$ and find an expression for x and y .)

3. **SVD for image compression.** In this problem, you will learn how SVD could be used in image compression.

- a. (5 pts) Compute the singular values of a pre-downloaded image (via the code provided below) and plot them. Note that the image has dimensions (height $\times$ width $\times$ channels), with channels = 3. You should compute and plot the singular values for each channel separately. Use a logarithmic scale.

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# feel free to use your favorite picture
# or work with the provided one
img = Image.open("disaster-girl.jpg", mode="r")
img = np.array(img).astype(np.int32)

plt.imshow(img)
plt.title('Original Picture')
plt.show()

# YOUR CODE BELOW
# extract and plot the singular values for each color channel
```

- b. (15 pts) Complete a function `compress` that performs SVD and truncates it (using k singular values/vectors). See the prototype below. Note that for 3-channel color images, you must split your image into channels and work with matrices corresponding to different channels separately. Plot 3 reconstructed images $\hat{X}$ of the original image X , using $\text{rank}(\hat{X}) = 5, 20, \text{ and } 50$.

```
def compress(image, k):
    """
    Perform svd decomposition and truncate it (using k singular
    ↪ values/vectors)

    Parameters:
        image (np.array):      input image

        k (int):                approximation rank

    -----
    Returns:
```

```

    reconst_matrix (np.array): reconstructed image

    s (np.array):          array of singular values
    """

    # YOUR CODE IS HERE

    return reconst_matrix, s

```

c. (15 pts) For this section, feel free to work only with one channel of the image. Plot the following figures.

- 1) How does the relative error ($\frac{\|\mathbf{X}-\hat{\mathbf{X}}\|_2}{\|\mathbf{X}\|_2}$) of approximation depend on the rank of approximation k ?
 - 2) How does compression rate depend on the rank of approximation in terms of storing information? ($(\mathbf{U}, \mathbf{V}$ sizes + k singular values to store) / total size of the original image)
- Share your observations from both figures. For example, what approximation rank (or number of the singular values k) would you choose to compress this image?

```

def rel_error(image, k):
    """
    Compute the relative error between the original image and its
    ↪ rank k approximation

    Parameters:
        image (np.array):    original image

        k (int):              approximation rank

    -----
    Returns:
        error (float):        relative error
    """
    # YOUR CODE IS HERE

    return error

def compression_rate(image, k):
    """
    Compute the compression rate of the image when approximated by
    ↪ its rank k SVD

    Parameters:
        image (np.array):    original image

        k (int):              approximation rank

    -----
    Returns:

```

```
    comp_rate (float):  compression rate
    """
    # YOUR CODE IS HERE

    return comp_rate
```